

OpenSpec: Spec-Driven Development nell'era degli Agenti AI

Un nuovo paradigma per la collaborazione tra Umani e Intelligenza Artificiale

Speaker: Matteo Baccan

L'Era degli Agenti AI

Lo sviluppo software sta subendo una trasformazione radicale grazie all'intelligenza artificiale. Siamo passati dalla scrittura del codice carattere per carattere all'orchestrazione guidata dagli intenti.

Gli agenti AI non sono più solo strumenti di autocompletamento, ma collaboratori attivi.

Il Problema: "Vibe Coding"

L'interazione naturale con gli LLM ha generato una pratica definita "vibe coding".

Consiste nel fornire istruzioni informali e non strutturate all'agente in una finestra di chat, sperando che l'output corrisponda all'idea originale.

I Sintomi del Vibe Coding

Questo approccio non strutturato porta rapidamente a:

- Allucinazioni architetturali.
- Scelte di librerie non coerenti con il progetto.
- Difficoltà estreme nell'effettuare una code review sensata.

La Perdita del Contesto

Nel vibe coding, i requisiti vivono esclusivamente nella cronologia della chat. Una volta chiusa la sessione, il contesto evapora. Non esiste alcuna documentazione persistente delle decisioni prese.

Il Limite della Finestra di Contesto

Più le conversazioni diventano lunghe, più gli agenti AI soffrono di "amnesia".
I dettagli iniziali dei requisiti vengono dimenticati o alterati, degradando drasticamente le prestazioni e l'accuratezza del codice generato.

Deriva dei Requisiti (Drift)

L'agente interpreta un prompt vago e aggiunge funzionalità non richieste ("mentre ci sono, ottimizzo questo").

Senza confini chiari, il progetto accumula funzionalità superflue e deviazioni dalla logica di business.

Il Debito Tecnico Nascosto

L'output del vibe coding genera un codice che funziona al momento, ma di cui nessuno (nemmeno l'agente futuro) conosce i presupposti esatti.

Il debito tecnico diventa difficilmente tracciabile e mantenibile nel tempo.

Dal Caos al Metodo

Un Nuovo Approccio: Spec-Driven Development

La Spec-Driven Development (SDD) inverte il paradigma: **la struttura prima del codice**.

L'intento deve essere formalizzato in specifiche chiare e leggibili dalle macchine prima che venga scritta una singola riga di codice.

Il punto non è generare più codice con l'AI, ma rendere l'intento esplicito, portabile e verificabile fin dall'inizio.

SDD: l'inversione della fonte della verità

Approccio Tradizionale (Chat-Driven)	OpenSpec (Spec-Driven)
Focus Output immediato (codice come verità).	Focus Definizione strutturata dell'intento (design come verità).
Ruolo della documentazione Onere post-sviluppo, spesso obsoleto prima del rilascio.	Ruolo della documentazione Istruzioni primarie ed eseguibili per l'AI. Guardrail architetturico.
Risultato Due fonti di verità divergenti: ticket e codice.	Risultato Il codice viene derivato in modo deterministico dalle specifiche.

In OpenSpec, la documentazione non rincorre il codice: lo precede e lo vincola.

Cos'è OpenSpec?

OpenSpec è un framework open-source progettato per la Spec-Driven Development.

Non richiede paywall o API key proprietarie per esistere come metodo, non è vincolato a un IDE specifico o a un singolo vendor AI.

È un sistema basato su file Markdown che funge da "controllo di versione per l'intento"

- **zero lock-in**
- specifiche portabili
- valore che resta nel repository.

OpenSpec: i tre pilastri

Pilastro 1: Brownfield-First ($1 \rightarrow n$)

Ottimizzato per l'evoluzione di codebase esistenti, non solo per prototipi greenfield ($0 \rightarrow 1$).

Permette di modificare logiche complesse in sicurezza, senza riscrivere tutto da zero.

Pilastro 2: Architettura Leggera

Markdown + Git come base operativa: niente infrastruttura pesante, niente database complessi.

Il metodo resta trasparente, versionabile e vicino al flusso reale del team.

Pilastro 3: Agnosticismo Totale

Zero lock-in verso IDE, modelli o vendor: il valore vive nelle specifiche, non nella piattaforma.

Se cambi agente o ambiente, il processo resta intatto perché l'intento rimane nel repository.

La Filosofia di OpenSpec

L'idea centrale è che la specifica sia la "fonte della verità", non il codice.

I documenti fungono da istruzioni eseguibili e vincolanti per gli agenti AI, non solo come suggerimenti o linee guida.

Il risultato non nasce da un'interpretazione libera: con OpenSpec c'è **solo intento deterministico**, espresso in un contratto scritto che l'agente deve seguire.

Living Documentation

Le specifiche sono mantenute in Git accanto al codice sorgente.
Non diventano mai obsolete perché si evolvono in parallelo al sistema, rendendo la documentazione un prodotto vivo dello sviluppo.

OpenSpec vs Strumenti di Project Management

I ticket nei sistemi tradizionali (come Jira o Linear) sono ottimi per gli umani, ma difficili da consultare in tempo reale dagli agenti AI.

OpenSpec porta le specifiche direttamente nel repository, dove l'agente opera.

Indipendenza dagli Strumenti AI

OpenSpec è universale. Supporta decine di agenti e ambienti di sviluppo diversi senza legarsi a un ecosistema proprietario.

Se un team decide di cambiare modello AI, estensione IDE o piattaforma, processi e specifiche rimangono intatti e validi.

Le specifiche vivono in Git: il valore resta tuo, non del vendor.

Ecosistema agnostico: zero lock-in come scelta strategica

Nodo centrale	Ecosistema collegabile
OpenSpec come livello stabile di intento e governance nel repository	Kilo Code, Cursor, Claude Code, Windsurf, GitHub Copilot e altri agenti compatibili

Capacità	Impatto operativo
Integrazioni intensive	Uso tramite CLI e slash commands nativi nei principali IDE/LLM, senza riscrivere le specifiche

Risultato: cambi strumento quando vuoi, senza perdere memoria progettuale né controllo sull'intento.

Approccio "Brownfield-First"

Molti framework AI sono ottimizzati per progetti nuovi (greenfield, $0 \rightarrow 1$).

OpenSpec brilla nei progetti esistenti (brownfield, $1 \rightarrow n$), dove l'integrazione di nuove funzionalità senza rompere le vecchie è cruciale.

La Meccanica di OpenSpec

L'Architettura del Framework

Tutto risiede in una singola directory radice all'interno del progetto:

```
openspec
```

Questa directory agisce come la memoria a lungo termine dell'agente AI.

La Separazione dello Stato

Il framework separa fisicamente:

1. La descrizione dello stato attuale del sistema.
2. Le proposte per le modifiche future.

Questa distinzione garantisce transizioni sicure e modifiche parallele.

L'architettura a due cartelle

L'Essere

`openspec/specs/`

La fonte della verità. Documentazione del comportamento attuale, organizzata per domini logici.

- `specs/auth/spec.md`
- `specs/payments/spec.md`
- `specs/ui/spec.md`

Il Divenire

`openspec/changes/`

Il workspace di modifica. Laboratorio isolato per le modifiche. Nessun conflitto con le specifiche principali finché non si archivia.

- `changes/add-oauth-login/`
- `proposal.md`
- `specs/...`

Il Cuore: La Directory **specs/**

Contiene la documentazione consolidata del comportamento attuale del software.

È la **"Source of Truth"** a cui l'agente deve attenersi prima di proporre o scrivere codice.

Organizzazione per Domini

I file in `specs/` sono organizzati per domini logici (es. `auth/`, `payments/`, `ui/`).

Ogni cartella ospita un file `spec.md` che descrive esattamente le capacità di quel comparto.

Il Workspace: La Directory **changes/**

Ospita le proposte di modifica. Ogni nuova feature o bug fix ottiene una propria cartella isolata.

Questo permette al team di preparare modifiche architetturali in modo pulito e strutturato.

Isolamento delle Modifiche

Lavorare in `changes/` previene i conflitti.

Più sviluppatori (e più agenti AI) possono preparare funzionalità diverse simultaneamente, senza inquinare la logica base finché non sono pronti.

Configurazione: Il File `config.yaml`

Funziona come la "costituzione" tecnica del progetto.

Sostituisce configurazioni frammentate fornendo un set di regole chiare, stack tecnologici e convenzioni di sviluppo.

Iniezione Attiva del Contesto

A differenza di un normale `README`, il contenuto di `config.yaml` viene **iniettato attivamente** nella finestra di contesto dell'agente in ogni interazione di pianificazione.

Garantisce che l'agente non dimentichi mai lo stack del team.

Governance attiva: come **config.yaml** guida ogni richiesta

config.yaml	Funzione	Effetto sull'agente
schema	Definisce workflow e struttura degli artefatti attesi	L'agente non improvvisa formati: produce output conformi
context	Fornisce architettura globale, stack, standard tecnici	Ogni piano parte dagli stessi vincoli di progetto
rules	Impone guardrail granulari per fase e artefatto	Riduce deviazioni, allucinazioni e scelte fuori policy

Risultato operativo: il file non resta passivo nel repository, viene iniettato nel prompt di pianificazione e rende la conformità tecnica sistematica.

Gli Artefatti della Pianificazione

Una cartella di modifica genera sempre un set standard di artefatti Markdown:

- `proposal.md`
- `design.md`
- `tasks.md`
- Le Delta Specs

Anatomia end-to-end di una proposta

Fase	File	Domanda chiave	Ruolo operativo
1	<code>proposal.md</code>	Perché / Cosa stiamo cambiando?	Definisce intento strategico e ambito lavori (business case iniziale).
2	<code>design.md</code>	Come lo realizziamo?	Fissa decisioni tecniche: architettura, flussi dati, librerie e vincoli.
3	<code>tasks.md</code>	Come eseguiamo in modo verificabile?	Scompone in task atomici e sequenziali, eseguibili dall'agente senza ambiguità.
4	Delta Specs	Cosa cambia nella Source of Truth?	Applica patch ai requisiti con sezioni <code>ADDED</code> , <code>MODIFIED</code> , <code>REMOVED</code> .

Flusso completo: `proposal.md` -> `design.md` -> `tasks.md` -> Delta -> consolidamento -> `specs/`

L'Artefatto 1: **proposal.md**

Cattura l'intento strategico. Risponde alle domande "perché stiamo facendo questa modifica?" e "qual è lo scopo principale?".

È il punto di allineamento iniziale tra umano e macchina.

L'Artefatto 2: **design.md**

È l'ancora tecnica. Delinea scelte di database, flussi di dati e specifiche librerie da utilizzare. Impedisce all'agente di deviare dai pattern stabiliti "immaginando" soluzioni creative ma errate.

L'Artefatto 3: **tasks.md**

Una checklist numerata di azioni da compiere.

Funge da registro di avanzamento. L'agente aggiorna le spunte in tempo reale mentre scrive il codice, garantendo totale trasparenza.

L'Importanza dei Task Atomici

I task in `tasks.md` devono essere sufficientemente piccoli da essere implementati dall'agente in autonomia, senza richiedere costanti chiarimenti o interventi dell'utente.

L'Artefatto 4: Delta Specs

La vera innovazione di OpenSpec. Invece di riscrivere l'intera specifica di sistema, l'agente crea una specifica "differenziale" (Delta) che mostra solo cosa cambierà.

Perché usare le Delta Specs?

In codebase enormi, rigenerare tutta la documentazione consumerebbe troppi token e tempo. Le Delta Specs si focalizzano solo sull'incremento funzionale, ottimizzando i costi e mantenendo alta l'accuratezza.

Anatomia di una Delta Spec

Le Delta Specs utilizzano intestazioni chiare per istruire il processo di fusione futuro:

1. **ADDED Requirements**
2. **MODIFIED Requirements**
3. **REMOVED Requirements**

Delta Tags: ADDED

Definisce comportamenti completamente nuovi.

Esempio: L'aggiunta di un sistema di autenticazione a due fattori in un progetto che prima aveva solo login base.

Delta Tags: **MODIFIED**

Descrive l'alterazione di comportamenti esistenti.

Richiede di riscrivere l'intero requisito aggiornato, garantendo che nessuna regola pregressa venga persa per distrazione dell'agente.

Delta Tags: REMOVED

Segnala funzionalità deprecate o da rimuovere.

Garantisce che il codice morto venga eliminato e che i vecchi test associati vengano disattivati correttamente.

Il Linguaggio dell'Intento

Scrivere Specifiche Efficaci

Le specifiche non devono essere istruzioni di programmazione step-by-step. Devono descrivere il comportamento osservabile del sistema dall'esterno. Sono contratti di business, non tutorial di codice.

Il linguaggio dell'intento: EARS + BDD

Asse	EARS (obbligazione)	BDD (esecuzione)
Scopo	Ridurre ambiguità nei requisiti	Rendere verificabili i comportamenti
Forma	Parole chiave normative: SHALL , MUST , SHOULD	Struttura scenario: GIVEN , WHEN , THEN
Domanda a cui risponde	"Cosa è obbligatorio?"	"Come si osserva il risultato?"
Esito pratico	Contratti chiari per l'agente	Test di accettazione derivabili

Regola operativa: prima definiamo il vincolo con EARS, poi ne proviamo l'esecuzione con BDD.

Sintassi EARS

OpenSpec adotta l'approccio EARS (Easy Approach to Requirements Syntax).
L'obiettivo è minimizzare l'ambiguità utilizzando parole chiave vincolanti per definire i livelli di obbligazione.

Le parole chiave: SHALL e MUST

Rappresentano comportamenti obbligatori e non negoziabili.

Se un test fallisce su un requisito **SHALL**, l'agente sa che l'implementazione è categoricamente errata.

La parola chiave: **SHOULD**

Indica una raccomandazione o un comportamento desiderato, ma con margini di flessibilità.
Permette all'agente di adattarsi a vincoli tecnici imprevisti durante la scrittura del codice.

Scenari Verificabili

Ogni requisito deve essere accompagnato da scenari pratici derivati dal Behavior-Driven Development (BDD).
Forniscono all'agente esempi inequivocabili di successo e fallimento.

Il Modello GIVEN / WHEN / THEN

- **GIVEN:** Il contesto iniziale (es. "Dato un utente non autenticato").
- **WHEN:** L'azione scatenante (es. "Quando visita la dashboard").
- **THEN:** Il risultato atteso (es. "Allora viene reindirizzato al login").

Dalla Specifica ai Test

Con una struttura GIVEN/WHEN/THEN chiara, l'agente AI è in grado di generare automaticamente i test unitari o e2e corrispondenti, chiudendo il ciclo della qualità del software.

Il Ciclo di Esecuzione

Il Ciclo Operativo (Workflow)

OpenSpec definisce una macchina a stati immutabile a tre fasi per ogni modifica:

1. Propose
2. Apply
3. Archive

Ogni transizione ha un output verificabile e impedisce di passare alla fase successiva senza allineamento.

La macchina a stati di OpenSpec

Fase	Obiettivo	Output della fase	Gate di passaggio
Propose	Allineare intento e ambito prima del codice	Cartella in <code>changes/</code> con <code>proposal.md</code> , <code>design.md</code> , <code>tasks.md</code> , Delta Specs	Revisione umana dell'intento
Apply	Implementare senza deviazioni dai vincoli	Codice + test aderenti a <code>design.md</code> e <code>tasks.md</code>	Verifica qualità su MUST/SHALL
Archive	Consolidare la modifica nella verità di sistema	Fusione Delta Specs in <code>specs/</code> + archivio storico della change	Stato aggiornato e audit trail persistente

Logica del ciclo: cattura presto il disallineamento (Propose), esegui in modo deterministico (Apply), consolida nella Source of Truth (Archive).

Fase 1: Proposta (Propose)

L'agente non scrive codice. Analizza la richiesta dell'umano e genera la cartella in `changes/` con la proposta, i task e le Delta Specs.

La Revisione dell'Intento

Questo è il momento chiave per lo sviluppatore umano.

Modificare un file Markdown errato richiede pochi secondi. Correggere un'architettura software errata dopo che è stata scritta richiede giorni.

Fase 2: Implementazione (Apply)

Solo quando gli artefatti sono approvati, l'agente inizia l'implementazione vera e propria. Legge i compiti in `tasks.md` e produce il codice rigorosamente entro i confini stabiliti nel `design.md`.

L'Esecuzione dell'Agente

Durante l'Apply, l'agente AI si trasforma in un mero esecutore.
Smette di indovinare le intenzioni e si limita a tradurre specifiche perfette in codice funzionante.

Gestione della Qualità

Prima di considerare chiusa la fase di implementazione, l'agente verifica il codice prodotto rispetto alle specifiche stabilite, correggendo autonomamente eventuali deviazioni dai requisiti MUST/SHALL.

Fase 3: Archiviazione (Archive)

Una volta che il codice funziona ed è stato testato, la modifica viene conclusa. Questo è il momento in cui l'intento temporaneo diventa documentazione permanente.

La Fusione della Verità

Il sistema unisce automaticamente le Delta Specs nei file originali della cartella `specs/`.
Il sistema si aggiorna: la nuova funzionalità fa ora ufficialmente parte della "Source of Truth" del progetto.

L'Archivio Storico

La cartella originale della modifica viene spostata in un archivio storico (es. ordinato per data).
Si preserva così l'audit trail delle decisioni architetturali per i futuri sviluppatori.

Funzionalità Avanzate: Schemi Personalizzati

Team diversi hanno processi diversi. OpenSpec permette di definire schemi personalizzati per generare artefatti specifici (es. audit di sicurezza obbligatori) prima dell'implementazione.

Sviluppo in Parallelo

In team numerosi, la SDD scala brillantemente.

La separazione in cartelle di modifica (**changes/**) consente di gestire backlog complessi senza corrompere la specifica di base del sistema.

Integrazione con Git Worktrees

OpenSpec si abbina perfettamente a Git Worktrees, permettendo agli agenti AI di testare modifiche isolate in rami paralleli prima di integrarle e archiviarle nella specifica principale.

Parallelismo e sincronizzazione

Sviluppo parallelo (Git Worktrees)	MCP (Model Context Protocol)
Branch multipli isolati sulla stessa codebase, senza contaminare la Source of Truth principale.	Collegamento tra stato OpenSpec e strumenti di tracking (es. Linear/Jira) tramite connettori MCP.
Orchestrazione di attività su feature diverse in contemporanea, con merge guidato dal ciclo Propose -> Apply -> Archive .	Backlog e stato reale del codice possono restare allineati, riducendo disallineamenti operativi.
Consolidamento finale nella Source of Truth con audit trail delle decisioni.	Aggiornamenti dei ticket più automatici e meno overhead amministrativo per il team.

Messaggio chiave: OpenSpec scala sia orizzontalmente, sia verticalmente

Sincronizzazione con il Project Management

A livello enterprise, OpenSpec può dialogare tramite protocolli MCP con strumenti di ticket come Linear o Jira.

Il backlog aziendale resta allineato con lo stato reale del codice, riducendo il lavoro amministrativo e i disallineamenti tra piano e implementazione.

Dal Team al Legacy

Il Problema del Codice Legacy

Come si introduce OpenSpec in un progetto esistente di 100.000 righe di codice senza documentazione?

Non riscrivendo tutto da zero.

Brownfield: adozione incrementale nei sistemi legacy

La scala dell'adozione	spec-gen (reverse engineering)
1. Non fermare il mondo Nessun bisogno di produrre specifiche upfront per milioni di righe.	Motore opzionale per accelerare la partenza su moduli complessi o poco documentati.
2. Sviluppo just-in-time Usa OpenSpec sulla prossima feature o sul bug critico, dove stai già intervenendo.	Analisi statica + pipeline LLM per estrarre regole di business dal codice esistente.
3. Accumulo organico La Source of Truth cresce naturalmente ad ogni ciclo Propose -> Apply -> Archive .	Generazione assistita di file spec.md retroattivi, da rifinire e validare con il team.

Principio guida

L'adozione parte dal lavoro reale di oggi; **spec-gen** è un acceleratore, non un prerequisito.

Adozione Incrementale

La strategia suggerita è documentare solo ciò che si tocca.
Se si deve modificare il modulo di pagamento, si documenta solo quello. Nel tempo, la documentazione crescerà organicamente.

Reverse Engineering delle Specifiche

L'uso di tool di supporto (come motori di `spec-gen`) permette di analizzare staticamente il codice sorgente esistente per generare architetture OpenSpec di base tramite AI, accelerando l'adozione.

Mitigazione dei rischi: 4 anti-pattern da evitare

Il cimitero delle proposte

Problema: dimenticare `Archive` e lasciare change mai consolidate.

Fix: integrare l'archiviazione nel workflow operativo e nei controlli di qualità.

Il micro-management dell'AI

Problema: scrivere dettagli implementativi nelle specifiche principali.

Fix: tenere il comportamento nelle spec e spostare il "come" in `design.md`.

Burocrazia per modifiche banali

Problema: usare una SDD completa per un cambio minimo e non rischioso.

Fix: applicare progressive rigor e mantenere un approccio leggero.

Ignorare la formattazione Delta

Problema: non usare `ADDED`, `MODIFIED`, `REMOVED` in modo strutturato.

Fix: rispettare i metadati Delta per merge puliti e consolidamento affidabile.

Anti-Pattern 1: Debito Speculativo

Il più grave errore è dimenticarsi di completare la fase di **Archive**.

Creare modifiche che rimangono perennemente in sospeso distrugge l'affidabilità della "Source of Truth", riportando il team al caos.

Anti-Pattern 2: Micro-management dell'AI

Scrivere dettagli implementativi nelle specifiche principali significa trasformare i requisiti in pseudo-codice.

Le specifiche devono descrivere il comportamento; il "come" va spostato in `design.md`, lasciando all'agente solo esecuzione e verifica.

Anti-Pattern 3: Burocrazia per modifiche banali

Usare una SDD completa per un cambiamento minimo e non rischioso introduce attrito inutile. Serve progressive rigor: più la modifica è piccola, più il processo deve restare leggero e orientato alla chiarezza.

Anti-Pattern 4: Ignorare la formattazione Delta

Saltare i tag **ADDED**, **MODIFIED**, **REMOVED** rompe il meccanismo di consolidamento e rende i merge meno affidabili.

Le Delta Specs non sono un vezzo sintattico: sono il contratto strutturale che permette fusioni pulite nella Source of Truth.

Perché Tutto Questo Conta

Impatto strategico: qualità del software e scalabilità umano-AI

Determinismo nell'esecuzione + Resilienza dell'intento = Scalabilità umano-AI

Precisione	Resilienza	Evoluzione del ruolo
L'esecuzione guidata da specifiche aumenta l'accuratezza al primo tentativo e riduce rilavorazioni e regressioni.	La conoscenza architetturale non muore nella chat: entra nel patrimonio del team e resiste al turnover.	Il valore umano si sposta dal "scrivere tutto" al garantire che il codice generato sia esattamente quello necessario.

Sintesi: quando l'implementazione è deterministica e l'intento rimane persistente, il sistema scala senza perdere qualità.

L'Impatto Strategico

La SDD trasforma l'incertezza dei prompt in un processo ingegneristico prevedibile.

- Accuratezza al primo tentativo elevatissima.
- Protezione del know-how aziendale.
- Scalabilità del team.

Conclusione

OpenSpec non riguarda lo scrivere meno codice, ma garantire che il codice generato dalle intelligenze artificiali sia **esattamente quello necessario**.
L'arte di definire l'intento è la competenza più preziosa del futuro.



Contatti

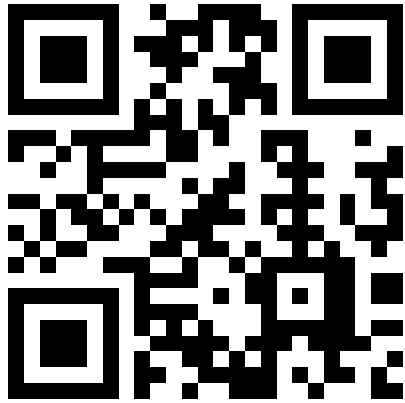
Grazie

<https://www.baccan.it>

"Smetti di chattare, inizia a governare."



Slide e link



baccan.it

<<https://www.baccan.it>>

Repo GitHub

<<https://github.com/matteobaccan/OpenSpec>>

Installazione su Windows

Per installare OpenSpec su Windows serve prima di tutto **Node.js 20.19.0 o superiore**.

```
node --version  
npm install -g @fission-ai/openspec@latest  
openspec --version
```

```
cd your-project  
openspec init
```

Link ufficiali:

- Installazione: <https://github.com/Fission-AI/OpenSpec/blob/main/docs/installation.md>
- Documentazione generale: <https://github.com/Fission-AI/OpenSpec/tree/main/docs>

Installazione OpenCode

```
npm i -g opencode-ai
```

Documentazione ufficiale:

- <https://opencode.ai/docs/it>

Installazione WezTerm

```
winget install wez.wezterm
```

Chi devo ringraziare per queste slide?

- Gemini: per la riformattazione
- Nano Banana Pro: per le immagini
- NotebookLM: per la prima scaletta e i riassunti dei podcast e video
- VSCode: per gestire il progetto GitHub
- Marp: per la presentazione